Engenharia de Software

5/3/2026

TP2

Velocidade e Qualidade com Estruturas de Dados e Algoritmos

Professor(a): Ricardo Pires Mesquita

Samuel Hermany
INSTITUTO INFNET

Link GitHub

https://github.com/faculdade-infnet/VI-1-velocidade-qualidade-com-algoritmos/tree/main/TP2
Obs. No GitHub consta além de um arquivo com código de cada questão, tem um outro com todas as questões e ordem, acredito que seja mais fácil para corrigir. tp1-todos-exercícios

1. Questão

2. Questão

3. Questão

4. Questão

Aqui busco identificar a letra ausente do alfabeto em uma string previamente informada.
Explicação:

- Criei uma função para verificar o que foi pedido acima.
- Criei uma array alfabeto com sub array **[letra, econtrada(True/False)]**, onde cada letra tem o valor default False e após ser encontrada seu valor é alterado para True.
- No "loop for" seguinte percorro a string informada, caractere por caractere, e então verifico se é uma letra de **a até z**, se a condição for verdadeira, então altero o valor default do sub array para True **[letra, True]**.
- Percorro o segundo "loop for" para criar dois arrays **[letras_encontradas]** e **[letras_nao_encontradas]** verificando se a posição 2 no sub array é True ou False.
- Retorno os dois arrays resultantes e planifico eles, separados os valores por **","**.
- Por fim, em tela são exibidas as duas string com as letras encontradas e as não encontradas.

5. Questão

Aqui busco retornar o primeiro caractere não repetido de uma string previamente informada.
Explicação:

- Criei uma função para verificar o que foi pedido acima.
- Criei um dicionário **[caracteres_contagem]** para armazenar os caracteres encontrados e o número de ocorrências **{caractere : nº ocorrência}**.
- No "loop for" seguinte percorro a string informada, caractere por caractere, e então eu verifico se ela já está contida no dicionário, se a condição for verdadeira, então altero o atribui +1 ao valor atual de contagem, senão, eu atribuo o novo caractere ao dicionário com valor inicial de 1.
- Percorro o segundo "loop for" dentro do dicionário e verifico o primeiro caractere único encontrado, armazenando ele e finalizando o loop.
- Retorno os dois itens (variável, dicionário).

- Por fim, em tela são exibidos os dois itens acima, com o primeiro caractere encontrado e o dicionário com a contagem de caracteres resultante.

6. Questão

O número de comparações tem a tendência de aumentar conforme **n** cresce, pois ele independe da ordem dos dados de entrada.
Já o número de trocas é menor quanto mais a lista ordenada estiver, havendo assim menos esforço computacional.
As variações de tempos são muito pequenas.

7. Questão

O número de comparações e de deslocamentos é menor quando a lista é ordenada ou quase ordenada, pois nos dois casos a lista já está quase no formato esperado, havendo assim menos esforço computacional.
Para n=30 a diferença de tempo foi pouca, mas a tendência é aumentar conforme **n** cresce.

8. Questão

FILA - pois o sistema atende as solicitações estritamente na ordem de chegada. A estrutura de dados que atende o que foi solicitado dentre as duas possíveis é fila, pois segue o princípio FIFO (First In, First Out), onde a primeira solicitação que entra é a primeira a ser atendida.
Operações envolvidas:

- Criar fila vazia
- Enfileirar - insere elemento no final da fila
- Desenfileirar - remove o primeiro item no inicio da fila
- Consultar o primeiro item
- Esvaziar fila, remove todos os itens
- Verificar se a fila está cheia
- Verificar se a fila está vazia

9. Questão

Neste exercício, construí uma pilha com capacidade fixa, com tratamento de erros, incluindo overflow (inserção em pilha cheia) e underflow (remoção em pilha vazia), sendo que para isso irei demonstrar o funcionamento com uma sequência de operações.
Explicação (os detalhes linha a linha estão comentados no código):

- Criei uma classe no primeiro bloco de código com as seguintes funções:
  - **init** - Iniciar/Criar nova pilha
  - **vazia** - verifica se pilha está vazia
  - **empilhar/push (adicionar)** - adiciona um novo item no topo da pilha
  - **desempilhar** - remove o item do top e retorna o item removido
  - **topo** - retorna o item do topo da pilha
  - **tamanho** - retorna o número de elementos da pilha
  - **esvaziar** - esvaziar pilha, remove todos os dados
  - **imprimir** - imprime todos os itens existentes na pilha
- Depois criei uma segundo bloco de código código para testar as funções acima:
  - imprimir - exibe os dados iniciais na pilha a cada adição
  - 1 - topo - exibe o item no topo da pilha

- 2 - tamanho - exibe o tamanho da pilha
- 3 - desempilhar - remove um item no topo da pilha
- imprimir - exibe os dados atuais na pilha
- 4 - empilhar - adiciona um item no topo da pilha
- imprimir - exibe os dados atuais na pilha
- 5 - topo - exibe o item no topo da pilha
- 6 - tamanho - exibe o tamanho da pilha
- 7 - empilhar - adiciona um item no topo da pilha
- imprimir - exibe os dados atuais na pilha
- 8 - topo - exibe o item no topo da pilha
- 9 - tamanho - exibe o tamanho da pilha
- esvaziar - limpar os dados da pilha
- 10 - tamanho - exibe o tamanho da pilha
- desempilhar - remove um item no topo da pilha
- imprimir - exibe os dados atuais na pilha

10. Questão

Neste exercício, construí uma fila com tamanho fixo, e controle de índices(início e fim).
Explicação (os detalhes linha a linha estão comentados no código):

- Criei uma classe no primeiro bloco de código com as seguintes funções:
  - init - Iniciar/Criar nova fila onde são definidos os ponteiros início e fim, além do tamanho inicial
  - vazia - verifica se fila está vazia
  - enfileirar/push (adicionar) - adiciona um novo item no final da fila
  - desenfileirar - remove o item do início da fila e retorna o item removido
  - início- retorna o item no inicio da fila
  - tamanho_fila - retorna o número de elementos da fila
  - esvaziar - esvaziar fila, remove todos os dados
  - imprimir - imprime todos os itens existentes na fila, início, fim e tamanho atual

11. Questão

12. Questão